Mini Colossal-AI

Distributed Parallelism Strategies from Scratch

Dinesh Chand
dineshchand@iisc.ac.in

Goutham P
gouthamp@iisc.ac.in

Meena Chidambaram
meenac@iisc.ac.in

Nadhiya R S
nadhiyar@iisc.ac.in

MTech-AI, IISc Bangalore

Problem Statement & Motivation

- Training modern LLMs (100M–billions of parameters) requires distributing computation across multiple GPUs
- Multiple parallelism strategies exist (DP, TP, PP, ZeRO) — each with different communication patterns and memory trade-offs
- The right strategy depends on both the **hardware topology** and the **model architecture**

What we did:

1. Built all four strategies **from scratch** using `torch.distributed` primitives
2. Composed them into hybrid configurations through a unified 3D plugin
3. Studied how **interconnect speed** and **axis-to-link mapping** affect throughput
4. Tested across **two architectures** (GPT-2 and T5) to isolate architectural effects

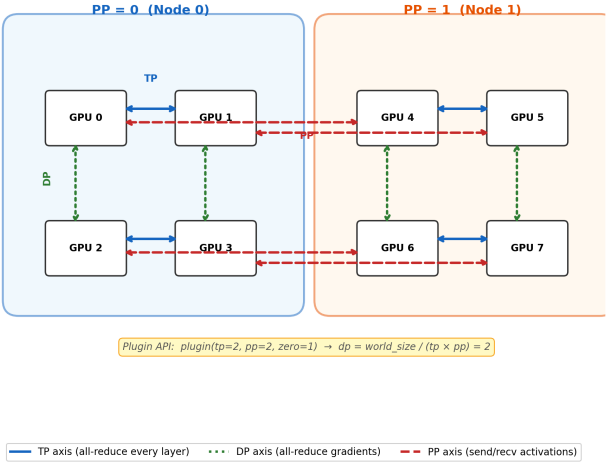
Methodology: Four Parallelism Strategies

1. **Data Parallelism (DP)**
   Replicate model on every GPU, average gradients via all-reduce after backward pass
2. **ZeRO Optimizer Sharding**
   Stage 1: partition optimizer states
   Stage 2: also partition gradients
   Cuts per-GPU memory by up to $N\times$
3. **Tensor Parallelism (TP)**
   1D column/row split (Megatron-LM style)
   2 all-reduces per transformer block
4. **Pipeline Parallelism (PP)**
   1F1B asynchronous schedule
   Only sends activations between adjacent stages

Axis	Communication
DP	All-reduce gradients (once per step)
TP	All-reduce partial sums (every layer, $2\times$)
PP	Send/recv activations (between neighbors)

All implemented using only `dist.send`, `dist.recv`, `dist.all_reduce`, `dist.all_gather`

Methodology: Unified 3D Hybrid Plugin



One plugin handles all combinations:

`plugin(tp=2, pp=2, zero=1)`
$\rightarrow$ dp = world_size / (tp $\times$ pp)

- Creates a 3D process group mesh
- Each GPU belongs to exactly one TP, PP, and DP sub-group
- All standalone components accept a process group argument
- No code duplication between standalone and hybrid modes

With 4 GPUs (Phase 2): 2 axes
With 8 GPUs (Phase 3): all 3 axes

How Placement Works: Reorder the Mesh Axes

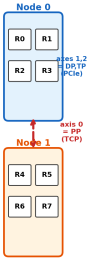
How Mesh(axis0, axis1, axis2) Maps Ranks to Nodes

Row-major order: axis 0 changes slowest → spans across nodes (slow TCP)

axis 2 changes fastest → stays within a node (fast PCIe)

Example: `Mesh(pp=2, dp=2, tp=2)` → axis 0 = PP, axis 1 = DP, axis 2 = TP

Rank	Coord (a0,a1,a2)	PP (axis 0)	DP (axis 1)	TP (axis 2)	Node
R0	(0, 0, 0)	0	0	0	Node 0
R1	(0, 0, 1)	0	0	1	Node 0
R2	(0, 1, 0)	0	1	0	Node 0
R3	(0, 1, 1)	0	1	1	Node 0
R4	(1, 0, 0)	1	0	0	Node 1
R5	(1, 0, 1)	1	0	1	Node 1
R6	(1, 1, 0)	1	1	0	Node 1
R7	(1, 1, 1)	1	1	1	Node 1



To switch placement, just change which axis is in position 0:

Good:	`Mesh(PP, DP, TP)`	*PP inter-node → lightest traffic on slow link*
Bad:	`Mesh(DP, PP, TP)`	*DP inter-node → heaviest traffic on slow link*
TP inter:	`Mesh(TP, PP, DP)`	*TP inter-node → frequent but small messages on slow link*

Methodology: Hardware Setup

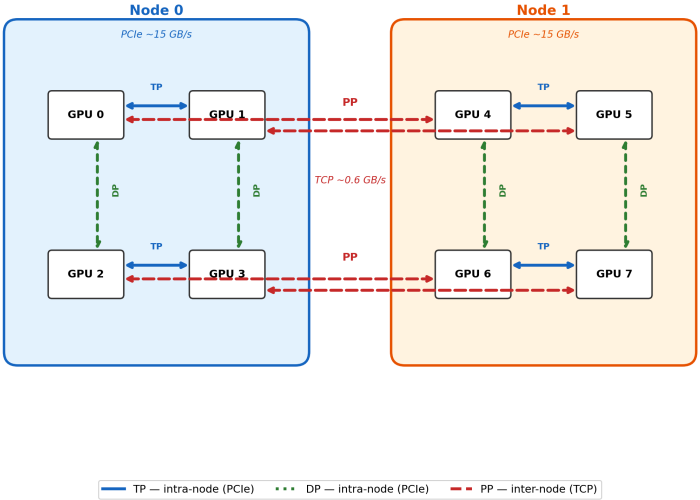
	Phase 2: g4dn.xlarge	**Phase 3: g4dn.12xlarge**
Nodes	4 instances	2 instances
GPUs / node	1× T4 (16 GB)	4× T4 (16 GB each)
Total GPUs	4	8
Intra-node link	N/A (single GPU per node)	PCIe (∼15 GB/s)
Inter-node link	TCP (∼0.6 GB/s)	TCP (∼0.6 GB/s)

Phase 2: Every GPU-to-GPU link is TCP
⇒ All strategies share the same slow link

Phase 3: 25× bandwidth gap
Fast PCIe within node, slow TCP between nodes
⇒ Which axis goes where matters

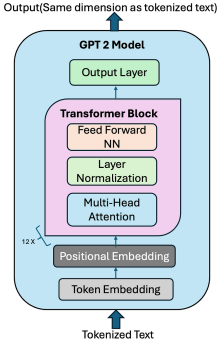
Methodology: Communication-Aware Placement

"Good" Placement: Heaviest communication stays on fastest link

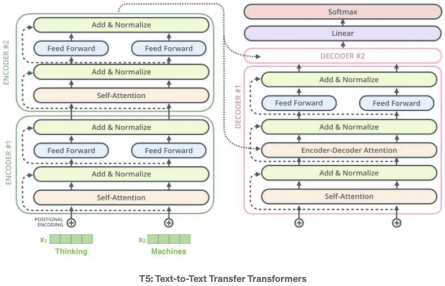


Three configs tested: **Good** (PP inter), **TP inter-node**, **Bad** (DP inter).

Methodology: Models Used



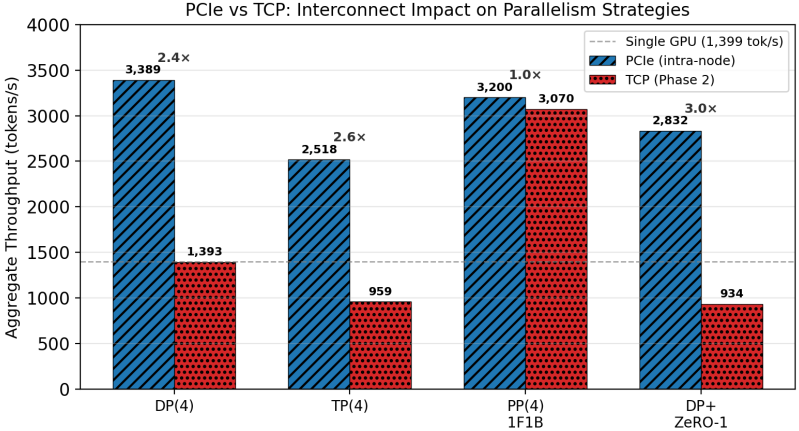
GPT-2 (Decoder-only)
Small: 117M, Medium: 354M, Large: 774M
All blocks identical ⇒ balanced pipeline



T5-base (Encoder-Decoder, 237M)
Decoder has extra cross-attention sublayer
Encoder lighter than decoder ⇒ pipeline imbalance

Results: Why Interconnect Speed Changes Everything

Setup: 4 GPUs on 1 node (PCIe) vs. 4 GPUs across 4 nodes (TCP) — GPT-2 Medium (354M)



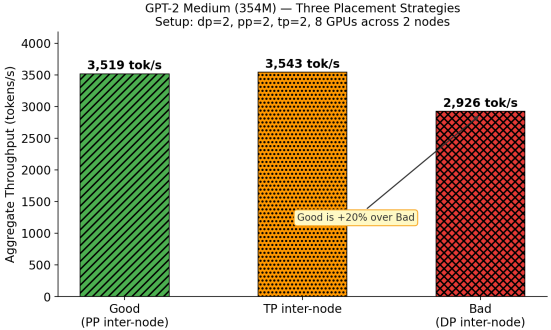
DP, TP, ZeRO: **2.4–3**× speedup on PCIe | PP: only **1.04**× (sends small activation tensors)

⇒ PP can tolerate the slow link; DP and TP cannot.

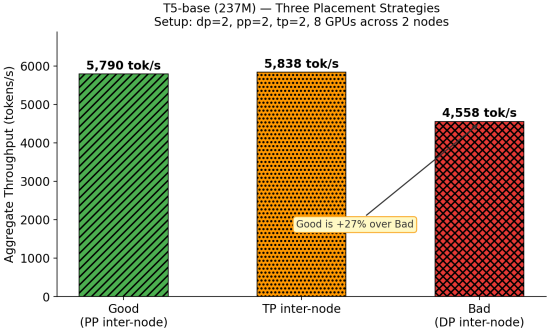
Results: Hybrid Placement — GPT-2 vs T5

Setup: dp=2, pp=2, tp=2 on 8 GPUs across 2 nodes

GPT-2 Medium (354M)



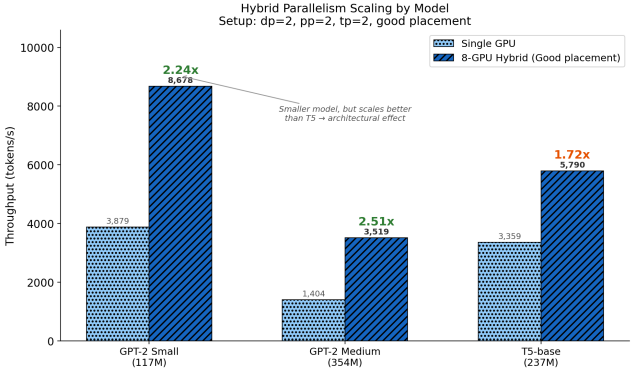
T5-base (237M)



- Both models: Good $\approx$ TP-inter $\gg$ Bad (DP inter-node). Bottleneck is **DP gradient sync** ($\sim$1.4 GB), not TP.
- GPT-2: Good is **+20%** over Bad. With ZeRO-1: **+26%** (3,044 vs 2,416).
- T5: Good is **+27%** over Bad — larger penalty because T5 has 60 vs 48 TP all-reduces (cross-attention) and is more **communication-bound**.

Results: Does Hybrid Scaling Depend on Architecture?

Setup: dp=2, pp=2, tp=2, good placement — single GPU vs 8-GPU hybrid



GPT-2 models: **2.2–2.5**× speedup.
T5-base: only **1.72**× speedup.

GPT-2 Small (117M) is *smaller* than T5 (237M) but scales *better*.

⇒ The gap is architectural:

1. **Pipeline imbalance**
   GPT-2: 4.07 vs 4.08 GB
   T5: 2.51 vs 3.87 GB (54%)
2. **Extra TP all-reduces**
   GPT-2 Med: 48 per fwd
   T5-base: 60 per fwd (+25%)

Key Findings

1. **Placement delivers 17–27% throughput gain**
   Keeping bandwidth-heavy axes (DP, TP) on the fast intra-node link consistently outperforms the reverse, across all models.
2. **DP overtakes TP as the placement bottleneck at low TP degrees**
   Standard expectation: TP is most sensitive (all-reduce every layer). At TP=2, individual TP messages are small (KB–MB) and TCP handles them fine. DP gradient sync ($\sim$1 GB+) saturates the slow link. At higher TP degrees (4, 8), TP would become the bottleneck again.
3. **Encoder-decoder models amplify the placement penalty**
   T5 sees +27% gain from good placement vs +20% for GPT-2. T5's extra cross-attention all-reduces make it more communication-bound, so slow-link overhead hits harder.
4. **Symmetric vs asymmetric architectures: 30–46% scaling gap**
   GPT-2's identical blocks give balanced pipeline stages and fewer all-reduces. T5's encoder-decoder split creates 54% memory imbalance and 25% more TP communication.

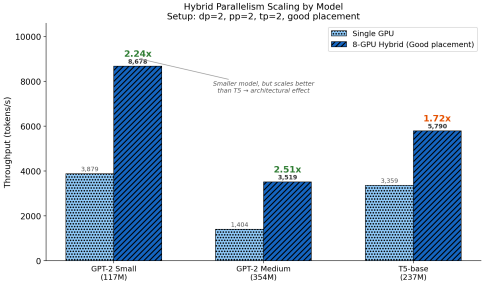
Summary

What we built:

- Four parallelism strategies from scratch
- Unified 3D hybrid plugin (DP×PP×TP)
- GPT-2 (Small / Medium / Large) implementation
- T5-base encoder-decoder implementation

What we showed:

- On a heterogeneous cluster, mapping axes to network tiers based on their communication volume gives 17–27% throughput gains
- DP is the most sensitive axis at low parallelism degrees
- Hybrid parallelism benefits depend on model architecture — symmetric GPT-2 scales 30–46% better than asymmetric T5



Same hardware, same config — architecture determines the scaling.

Questions?

Dinesh Chand Goutham P Meena Chidambaram Nadhiya R S

MTech-AI, IISc Bangalore